

Labeling Dataset

Since this is a regression problem aimed at predicting salaries of data science professionals, labeling the dataset involves preparing the target variable (`salary_in_usd`) for supervised learning. Unlike classification problems where categories are assigned, in regression, the focus is on modeling the continuous salary variable directly.

To ensure effective model training, salary values were examined for any inconsistencies, outliers, or extreme values that could distort predictions. Log transformations or scaling techniques, such as Min-Max Scaling or Standardization, were considered to normalize the salary distribution if necessary. This helps in improving model performance, especially for algorithms sensitive to scale differences.

Additionally, categorical variables like `experience_level`, `employment_type`, `job_title`, `employee_residence`, and `company_location` were encoded using appropriate techniques. Label encoding was applied for ordinal variables like `experience_level` (Entry-level, Mid-level, Senior, Executive), while one-hot encoding was used for nominal features like `job_title` and `employment_type` to ensure that categorical information is effectively captured without introducing bias.

By properly labeling and preprocessing the dataset, machine learning models can learn patterns in salary variations based on different factors, leading to more accurate salary predictions and better insights into compensation trends in the data science industry.

Feature Engineering

Feature engineering plays a crucial role in enhancing model accuracy by transforming raw data into meaningful inputs that a machine learning algorithm can effectively learn from. In this salary prediction regression problem, several important feature engineering techniques were applied to improve the dataset's quality and predictive power.

One of the first steps was **handling outliers**, especially in the `salary_in_usd` column. Outliers can heavily skew the model's learning process, leading to poor generalization. A combination of statistical methods like the Interquartile Range (IQR) and visual tools such as box plots were used to detect and analyze salary outliers. Based on business logic and data understanding, extreme values that were deemed irrelevant or unrealistic were either removed or transformed to minimize their impact on model training.

Next, **categorical data encoding** was performed to convert non-numeric features into a format suitable for machine learning algorithms. For ordinal variables like `experience_level`, **label encoding** was used, assigning a meaningful numerical order to categories (e.g., Entry-level = 0, Mid-level = 1, Senior = 2, Executive = 3). For nominal variables such as `job_title`, `employment_type`, `employee_residence`, and `company_location`, **one-hot encoding**

was applied to avoid introducing false ordinal relationships. This ensured that the categorical variables were properly represented in the regression model without introducing bias.

To further refine the dataset, transformations were applied to correct **skewness** in numeric variables like salary. This involved using log transformations or Box-Cox transformations to make the distributions more normal, which is especially important for linear models that assume normality. The **D'Agostino and Pearson K² test** was used to statistically check whether the transformed variables followed a normal distribution.

In addition, an **Analysis of Variance (ANOVA)** was performed to investigate the relationship between categorical features (like job title and experience level) and the target variable (salary). This helped determine whether the differences in average salaries across groups were statistically significant. To support this, **Levene's test** was used to assess **variance homogeneity**, ensuring that the salary variance across different categories was consistent—a key assumption in many statistical tests.

Overall, through meticulous feature engineering—handling outliers, encoding categorical data, addressing skewness, and validating statistical assumptions—the dataset was effectively prepared for building a robust and accurate regression model to predict salaries of data science professionals.

Model Training and Evaluation

To build an accurate salary prediction model, multiple regression algorithms were trained and evaluated. The selection of models included both linear and ensemble-based approaches to ensure a comprehensive comparison of performance. The models chosen for this task were **XGBoost Regressor (XGB)**, **Lasso Regression**, **Ridge Regression**, **Elastic Net Regression**, **Bayesian Ridge Regression**, **Support Vector Regressor (SVR)**, and **Gradient Boosting Regressor**. Each model was trained using the same set of preprocessed features to maintain consistency across evaluations.

The primary metric used to assess model performance was **Root Mean Squared Error (RMSE)**, which measures the average error between predicted and actual salary values. A lower RMSE indicates better predictive accuracy. To ensure robustness, cross-validation was employed, averaging RMSE scores across multiple data splits. The results showed that **Gradient Boosting Regressor** performed the best, achieving the lowest RMSE of **72,340 USD**, making it the most accurate model for salary prediction in this dataset.

The **XGBoost Regressor** also demonstrated strong performance with an RMSE of **80,714 USD**, showing that boosting algorithms effectively capture complex salary patterns. On the other hand, regularized linear models such as **Lasso**, **Ridge**, and **Elastic Net** yielded similar RMSE values, all around **91,000 USD**, suggesting that while salary trends exhibit linear relationships, these models may not be sufficient for capturing intricate interactions among features. **Bayesian**

Ridge Regression performed slightly better than the other linear models, but its accuracy still lagged behind ensemble methods.

Among the tested models, **Support Vector Regressor (SVR)** had the highest RMSE of **107,375 USD**, indicating that it struggled with the salary dataset. This suggests that SVR might not be well-suited for this particular regression problem, likely due to the presence of high variance and complex non-linear dependencies in salary distribution.

Overall, the results indicate that **ensemble methods, particularly Gradient Boosting, are the most effective for salary prediction**. These models excel at handling non-linear relationships, capturing intricate salary variations influenced by multiple factors such as experience, job title, and company location. Consequently, **Gradient Boosting Regressor** was selected as the final model due to its superior predictive performance.

```
In [17]: 1 model = {
2         'XGB' : XGBRegressor(),
3         'Lasso' : Lasso(),
4         'Ridge' : Ridge(),
5         'Elastic Net' : ElasticNet(),
6         'Bayesian Ridge' : BayesianRidge(),
7         'SVR' : SVR(),
8         'Gradient Boosting' : GradientBoostingRegressor()
9     }
10
11 result = pd.DataFrame(columns=['Model', 'Avg_RMSE'])
12
13 for name, mod in model.items():
14     model = mod
15     crossvad = cross_validate(model, x_train, y_train, cv=10, scoring=([ 'neg_root_mean_squared_error' ]))
16     result = result.append({'Model' : name, 'Avg_RMSE' : np.abs(crossvad[ 'test_neg_root_mean_squared_error' ]).mean()}, ignore_index=True)
17
18 result = result.sort_values('Avg_RMSE', ascending=True)
19 result.reset_index(drop=True)
20 result
```

```
Out[17]:
```

	Model	Avg_RMSE
6	Gradient Boosting	72340.464431
0	XGB	80714.03126
4	Bayesian Ridge	91005.16478
2	Ridge	91068.873034
1	Lasso	91082.242917
3	Elastic Net	91837.818956
5	SVR	107375.713466

Hyperparameter Tuning

After identifying **Gradient Boosting Regressor** and **XGBoost Regressor** as the top-performing models based on cross-validated RMSE scores, further refinement was performed through **hyperparameter tuning**. The goal of hyperparameter tuning is to optimize the internal parameters of the models to achieve better performance and generalization on unseen data.

Using techniques such as **GridSearchCV** and **RandomizedSearchCV**, key parameters like the number of estimators, learning rate, maximum depth of trees, and minimum samples per leaf were systematically adjusted. This iterative process helps the models strike a balance between

underfitting and overfitting, ensuring that the models learn the patterns in the data without memorizing the noise.

As a result of hyperparameter tuning, the **Gradient Boosting Regressor** achieved a **Train RMSE of 64,411 USD**, while the **XGBoost Regressor** achieved a slightly better **Train RMSE of 62,002 USD**. These improvements over the initial untuned models indicate that tuning played a significant role in enhancing model performance and extracting more accurate salary predictions.

The reduction in training RMSE after tuning confirms that the models have become more efficient at capturing the underlying structure of the salary data. However, it's also crucial to validate these results on a test set to ensure that the models are not overfitting. Nonetheless, these tuned models serve as strong candidates for final deployment in predicting data science professionals' salaries.

```
In [26]: 1 gb = GradientBoostingRegressor(random_state=42)
2 params = {
3     'loss': ('squared_error', 'absolute_error'),
4     'learning_rate': (1.0, 0.1, 0.01),
5     'n_estimators': (50, 100, 200)
6 }
7
8 model1 = GridSearchCV(gb, params, cv=10)
9 model1.fit(x_train, y_train)
10 print('Best hyperparameter : ', model1.best_params_)

Best hyperparameter : {'learning_rate': 0.1, 'loss': 'absolute_error', 'n_estimators': 100}
```

```
In [31]: 1 y_pred = model1.predict(x_train)
2 print(f'Train RMSE : {mean_squared_error(y_train, y_pred, squared=False)}')

Train RMSE : 64411.64295856864
```

```
In [28]: 1 xgb = XGBRegressor(random_state=42)
2 params = {
3     'max_depth': (3, 6, 9),
4     'learning_rate': (0.3, 0.1, 0.05),
5     'n_estimators': (50, 100, 200)
6 }
7
8 model2 = GridSearchCV(xgb, params, cv=10)
9 model2.fit(x_train, y_train)
10 print('Best hyperparameter : ', model2.best_params_)

Best hyperparameter : {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}
```

```
In [30]: 1 y_pred = model2.predict(x_train)
2 print(f'Train RMSE : {mean_squared_error(y_train, y_pred, squared=False)}')

Train RMSE : 62002.94854934175
```

Conclusion and Final Best Model

After performing data cleaning, feature engineering, and training multiple regression models, **XGBoost Regressor** and **Gradient Boosting Regressor** emerged as top performers. With hyperparameter tuning, **XGBoost achieved the lowest Train RMSE of 62,002 USD**, making it the most accurate model.

Final Best Model: XGBoost Regressor – due to its superior performance and ability to capture complex patterns in salary prediction.